Step 1 — The Security Reviewer on a Diff — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

This project is deliberately lighter on hand-holding than Projects 1-5 — try genuinely hard before reading Hint 2.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and the one case that matters

Hint 1 — The idea, and the exact pieces

The smallest useful piece: a Security reviewer that reads a diff's **added lines** and flags secrets and unsafe patterns. Build only what that needs:

- `src/diff.py` — `parse_diff(text)` returns a list of `{path, added}`, one per changed file. Only the lines that start with `+` (and not `+++`) are "added"; the `+++ b/<path>` line gives the file path.
- `src/schemas.py` — `Finding(severity, category, message)`, a Pydantic model.
- `src/agents/security_reviewer.py` — `run_security(files)` scans each file's added lines and returns `Finding`s.
- `src/config.py`, `src/exceptions.py`, `src/utils/logger.py`, `src/models/llm.py` — the standard setup from every project.
- `main.py` — run it on a dirty diff and a clean one, print findings.

Hint 2 — The plan, and the one case that matters

Keep the scanner **deterministic** — regex patterns, not a prompt. That's what makes it testable:

```
SECRET_PATTERNS  = [AWS key (AKIA...), password|secret|api_key = "..."]
UNSAFE_PATTERNS  = [eval(, subprocess(..., shell=True), pickle.loads()]

run_security(files):
    findings = []
    for file in files:
        for line in file["added"]:
            if a secret pattern matches  -> Finding(high,   "security", ...)
```

```
            if an unsafe pattern matches -> Finding(medium, "security", ...)
    return findings
```

The case that matters is the **clean** diff: a reviewer only ever tested on bad input will happily flag good code. So `test_security` has two cases — a planted diff (secret + `eval` flagged) **and** a clean diff (nothing flagged). The whole file set is in the Solution.